Contract Guided Requirement to Multiphysics Compilation for Electronic Design Automation

Anonymous submission

***Abstract*—*Engineering intent does not directly specify the physics interfaces, couplings, boundaries, and validation evidence needed for an executable multiphysics model. We present a contract guided compiler that maps requested observables to COMSOL dependencies, blocks missing or contradictory requests, and permits execution only after reviewable static and physical gates. On 24 project authored development tasks, the complete method made 24/24 correct gate decisions and 22/24 exact physics and coupling selections; direct and retrieval language model baselines each made 16/24 correct gate decisions. Two frozen EDA scenarios then completed fresh post freeze COMSOL solves. We release a frozen protocol for a 24 task, eight family external evaluation; its independent results are intentionally withheld until collection is complete.*

***Index Terms*—*EDA automation, multiphysics simulation, COMSOL, requirement contracts, physical validation, language model agents*

I. INTRODUCTION

Electronic products increasingly require coupled electrical, thermal, structural, fluid, acoustic, and transport analyses. A designer may ask for peak temperature, voltage drop, thermally induced displacement, pressure drop, or a resonance frequency without naming the numerical interfaces needed to compute that observable. Translating the request into a solver model therefore requires decisions about physics, couplings, geometry, materials, boundary conditions, studies, outputs, mesh, and acceptance tests. A plausible script can still be unsafe when decisive information is absent or internally inconsistent.

Language model agents have begun to automate EDA scripts and finite element inputs [1]–[5]. Their tool calls and execution feedback improve over single pass generation, but solver completion does not establish that the selected model answers the requested engineering question. For multiphysics EDA, a system must also decide when a request is insufficient to execute, preserve that decision across code generation and repair, and record physical evidence independently of the model that proposed the script.

We study the following falsifiable question: can an explicit requirement and dependency contract reduce unsafe progression from engineering intent to COMSOL execution while retaining complete requests? The proposed workflow converts requested observables into typed interface and coupling dependencies, checks interface specific completeness, and exposes a blocking decision before code generation. Static, execution, and physical validation gates then determine whether a generated model may support an EDA decision.

The paper makes three bounded contributions:

- A contract guided dependency compiler that links requested observables to COMSOL interfaces, couplings, studies, critical inputs, and clarification reasons.
- A local authority model in which language model output is advisory, while deterministic gates control construction, execution, repair, evidence, and surrogate release.
- A frozen evaluation structure that separates project authored development evidence from a sealed external, family disjoint experiment with repeated language model runs.

The current measured comparison uses 24 project authored development tasks and two post freeze COMSOL EDA scenarios. It supports debugging and controlled comparison, not an independent generalization claim. The external evaluation protocol is frozen before data acquisition; its result table remains absent until the required tasks and repetitions are complete.

II. CONTRACT GUIDED COMPILATION

A. Typed requirement contract

Every request is converted into a versioned contract containing the engineering objective, design and operating variables with units and ranges, geometry and material assumptions, physics interfaces, couplings, boundary and initial conditions, study, outputs, validation rules, evidence level, and unresolved fields. The contract is an execution boundary. A generated builder cannot add a decisive load, material property, grounding condition, heat transfer boundary, support, inlet, or study interpretation that the user did not provide or approve.

B. Observable to dependency graph

The compiler starts from requested observables rather than surface keywords. Temperature creates a heat transfer dependency; voltage and current require electric currents; displacement and stress require solid mechanics; pressure drop requires flow; acoustic pressure and resonance create acoustic and frequency study dependencies. Joint observables activate candidate couplings: electrical and thermal goals imply electromagnetic heating, thermal and structural goals imply thermal expansion, and flow temperature goals imply nonisothermal flow. Each edge records its trigger, required evidence, and validation level.

For a normalized request r, the compiler returns a tuple containing the interface set I, coupling set K, study S, blocking decision B, clarification set Q, and an evidence trace E. Evaluation uses exact set equality for I and K rather than partial credit, exact equality for S, and a binary comparison for B. The evidence trace identifies whether each interface came from direct text evidence or from an observable dependency. This representation separates a technically plausible interface list from an executable contract: the same interface prediction can be scored as unsafe when the boundary evidence needed to solve it is absent.

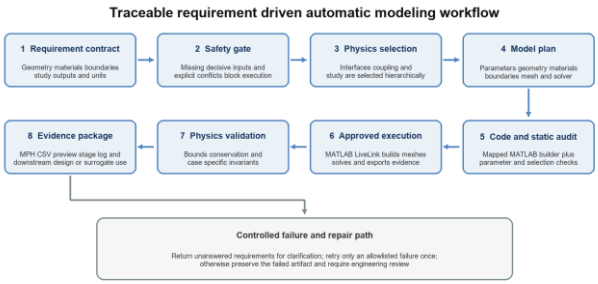


Fig. 1. Requirement, contract, execution, and evidence flow.

C. Missing and conflicting evidence

A negation aware layer separates available evidence from clauses marked unavailable, unknown, unspecified, deferred, prohibited, or contradictory. Interface specific checks then decide whether execution is allowed. An electrothermal request may be blocked for absent current magnitude, ground, thermal environment, or material data. A thermal stress request may be blocked for absent mechanical support or reference temperature. A request for pressure drop that forbids flow or for a resonance frequency under a stationary study is recorded as an explicit conflict. The output includes reason codes and clarification questions instead of invented defaults.

TABLE I. CONTRACT FIELDS AND AUTHORITY

Field	Source	Effect
Objective and output	Request	Creates dependency edges
Physics and coupling	Compiler	Requires traceable evidence
Boundary and material	Request or approval	Blocks when decisive data are absent
Validation rule	Frozen contract	Controls solved and passed labels
Repair	Diagnosed allowlist	Cannot change contract semantics

D. Ten auditable stages

Accepted contracts pass through model definition, geometry, materials, physics, boundaries, mesh, study, outputs, static audit, and approved generation. Execution records server connection, model tree construction, selection checks, meshing, study completion, result export, MPH save, and preview generation. Artifacts are fingerprinted. A single worker lock avoids concurrent LiveLink process trees, and a hard timeout terminates stalled MATLAB children.

E. Repair boundary

One bounded repair may address a diagnosed syntax, tag, selection, expression, or solver setting fault. Geometry dimensions, material values, loads, boundary values, selected interfaces, coupling removal, validation thresholds, and the contract itself require renewed review. Tool success after an unauthorized semantic change is therefore not counted as a repaired success.

III. EDA EXECUTION AND PHYSICAL EVIDENCE

A. Physical acceptance

Solver convergence is necessary but insufficient. The platform evaluates case specific invariants: electrical current continuity, Joule power transfer, thermal energy balance, structural reaction and finite response, flow mass balance and pressure drop, acoustic reference frequency and wavelength resolution, and output finiteness. A model receives a solved label only when its required numerical and physical checks pass. MPH, CSV, logs, previews, code, contracts, reports, and hashes remain linked to the same task identifier.

The validation contract distinguishes construction, numerical, and physical evidence. Construction evidence confirms that the requested model tree and selections exist. Numerical evidence confirms meshing, convergence, and finite outputs. Physical evidence applies invariants and declared thresholds. A task may therefore be constructed but not solved, solved but not physically accepted, or accepted for a named decision while remaining outside any product qualification claim. This state model prevents a successful process exit code from overwriting a failed balance or a missing output.

B. Frozen EDA scenarios

Two machine readable EDA scenarios define decision variables, constraints, outputs, physics, stopping conditions, and selection rules before new COMSOL execution. The busbar power delivery scenario compares two nominal three dimensional designs under a 160 A terminal current. Both satisfy current and energy balance, maximum temperature, voltage drop, current density, and geometry gates. The frozen lower loss rule selects a design with 318.388 K peak temperature, 11.698 mV drop, 1.872 W Joule loss, and 0.2231 kg copper mass. The second point is retained as a lower mass tradeoff.

The MEMS thermal actuator scenario couples electric currents, heat transfer, and structural response. A four point voltage and surface cooling grid contains three feasible points. The frozen maximum displacement rule selects 5.25 V and 22500 W/(m²·K), producing 507.377 K, 1.120 µm displacement, and 15.604 mW. The results support the named choices at the tested nominal points. They do not establish process corner robustness or fabricated device qualification.

TABLE II. POST FREEZE EDA DECISIONS

Scenario	Fresh points	Decision evidence
Busbar	2	2 feasible; lower loss selected
MEMS actuator	4	3 feasible; maximum displacement selected

C. Dual execution and local authority

Six mature representative templates spanning heat transfer, Joule heating, solid mechanics, nonisothermal flow, pressure acoustics, and species transport have completed current workstation MATLAB LiveLink runs with saved MPH and physical evidence. The same verified models were exported as template specific Java sources, compiled with the COMSOL 6.4 toolchain, reconstructed, solved, and saved as new MPH artifacts. This dual path checks that the evidence is not tied to one scripting front end. It does not prove portability to another workstation, which still requires a destination installation test.

D. Assistant boundary

Offline parsing, contracts, generation, execution, and evidence remain local. A configured language model may explain a request or analyze a redacted failure, but it cannot mark a stage approved, start an unreviewed solve, modify

evidence status, or release a surrogate. This separation allows language model assistance without assigning it authority over claims that depend on deterministic artifacts.

E. Artifact and replay contract

Each accepted execution writes the normalized request, contract version, generator version, COMSOL and MATLAB path configuration, source hash, stage log, result table, MPH hash, and validation report. Replay first verifies frozen inputs and source hashes, then runs in a new output directory. Historical MPH files remain inspectable evidence but are not counted as a new workstation acceptance. Compact JSON and CSV artifacts are included in the reproducibility package; large binaries remain referenced by hash and must be regenerated where licensing permits.

IV. CONTROLLED DEVELOPMENT EVALUATION

A. Tasks and methods

The controlled development set contains 24 requirements from eight electronic device families, with eight complete, eight critical information missing, and eight explicit conflict tasks. Four methods receive the same text and output contract: a direct language model, a retrieval augmented language model, the existing selector, and the complete contract dependency compiler. The two language model methods use fresh sessions and produce structured outputs. This set was authored inside the project and is labelled development evidence throughout.

TABLE III. PROJECT AUTHORED DEVELOPMENT RESULTS

Method	Physics exact	Gate correct	False safe / block
Direct LLM	23/24	16/24	0 / 8
Retrieval LLM	23/24	16/24	0 / 8
Existing selector	21/24	19/24	2 / 3
Contract dependency	22/24	24/24	0 / 0

Both language model baselines match 23/24 physics and coupling labels but block all eight complete requests, yielding 16/24 correct gate decisions. The complete method matches 22/24 interface and coupling sets and makes 24/24 correct gate decisions. The existing selector makes two false safe and three false block decisions. The complete method therefore improves the safety decision under this controlled set while leaving two physics label errors visible.

B. Confidence and paired comparison

The complete method gate accuracy is 1.000 with a Wilson 95% interval of [0.862, 1.000]. Zero false safe observations among 16 blocked tasks still has a Wilson upper bound of 0.194, which prevents a zero risk interpretation. The direct and retrieval gate accuracy is 0.667 with interval [0.467, 0.820]. The exact McNemar comparison between the complete method and each language model baseline is $p=0.007812$ for gate correctness. These values characterize the frozen development set only.

C. Ablation

Removing negation aware completeness reduces correct gate decisions from 24/24 to 16/24 and creates eight false safe decisions. Removing the safety gate reduces correctness to 8/24 and creates 16/16 false safe decisions. The ablation supports the need for explicit negative evidence and an execution block. It cannot establish performance on independently written requirements.

D. Evidence accounting

All 96 method rows are measured and retained with prompts, structured outputs, diagnostics, timing, interaction counts, and validity status. Invalid language model outputs would count as method failures rather than disappearing from the denominator. Development and submission gates are separate machine readable fields so a complete local experiment cannot be relabelled as independent publication evidence.

E. Controlled method conditions

The four methods share the same requirement text, expected JSON fields, adjudicated labels, and scoring code. Direct and retrieval runs use the same model and reasoning budget; retrieval alone receives the frozen template catalog and contract schema. Deterministic methods run without network access. Method specific raw responses are preserved before parsing, and the parser rejects missing fields or values outside the contract. Runtime is reported separately because local deterministic execution and hosted language model latency are not directly comparable as computational costs.

F. Observed failure taxonomy

The development errors fall into three actionable categories. A physics completion error omits or adds an interface or coupling. A study error selects a study inconsistent with the adjudicated target. A safety error either permits a missing or conflicting request or blocks a complete request. This separation matters because a conservative method may avoid false safe decisions by refusing useful tasks, as the two language model baselines do here. The complete method removes observed safety errors but still requires analysis of the two interface mismatches before any broader claim.

V. INDEPENDENT PROTOCOL AND RELATED WORK

A. Prospective P3 protocol

The independent evaluation protocol was frozen before external acquisition. It requires 24 requests from eight new device families. For each family, a project external author supplies one complete, one missing, and one conflict request; a different person adjudicates physics interfaces, couplings, study, and block decision without method outputs. The family labels must not overlap any declared development, validation, or P2 family. The accepted task file and protocol are sealed by SHA 256 before any method run.

Direct and retrieval language model methods each run three independent sessions per task. The two deterministic methods each run once, producing 192 expected observations. The frozen evaluator reports task consensus, per family exact physics and study accuracy, gate accuracy, false safe and false block counts, Wilson intervals, exact McNemar tests, paired bootstrap completion time intervals, repeat agreement, and a failure taxonomy. Partial runs, synthetic rows, project authored replacements, and post unseal method changes do not pass the main result gate.

TABLE IV. FROZEN INDEPENDENT EVALUATION CONTRACT

Item	Requirement	Current

Tasks	24 external	0
Families	8 disjoint	0
Task types	8 complete, 8 missing, 8 conflict	Pending
LLM repeats	3 + 3 per task	0
Total observations	192	0

At manuscript generation time the infrastructure gate passes, but accepted external tasks are 0/24, new families 0/8, and observations 0/192. No independent P3 result is reported. This explicit absence prevents a development table from being presented as generalization evidence. The final submission gate remains open until the sealed result exists.

B. Related EDA agents

ChatEDA orchestrates RTL to GDSII scripts from natural language [1]. LayoutCopilot maps analog layout intent to commands and suggestions [2]. EDAid uses multiple agents for long EDA tool chains [3], AutoEDA standardizes microservice interactions [4], and FlowFixer EDA accepts repairs only after tool reruns and quality checks [5]. These systems establish the relevance of tool grounded agents in EDA. Our scope differs in the compiled object: observable dependencies, multiphysics couplings, critical engineering evidence, and physical acceptance rather than digital flow scripts or layout commands.

C. Simulation agents

MooseAgent uses retrieval and iterative verification to generate MOOSE inputs [6]. ALL FEM combines fine tuned models, specialized agents, and runtime feedback for FEniCS [7]. PDE Agents studies graph augmented reasoning and execution [8], while MOOSEnger combines repository retrieval, executable diagnosis, and persistent memory [9]. We do not claim broader code generation. The intended distinction is a bounded COMSOL dependency compiler, a deterministic block decision, and explicit separation between advisory generation and authority to solve, validate, and release evidence.

D. Position within DAC

The primary fit is AI based design flow automation because the system translates an engineering request into a guarded electronic and multiphysics design workflow. The secondary fit is benchmark and model evaluation because the contribution includes task contracts, method controls, confidence intervals, repeated runs, and explicit evidence tiers. The work is not an RTL generator, a physical design optimizer, or a general purpose simulation chatbot. Its EDA value is the prevention and diagnosis of invalid multiphysics progress before a designer commits compute time or accepts a nominal design decision.

VI. DISCUSSION AND CONCLUSION

A. What the evidence supports

The measured results show that exact interface selection and safe progression are distinct objectives. The language model baselines obtain high interface accuracy while blocking every complete request. The existing selector is less conservative but permits two blocked requests. The contract dependency method removes both observed gate error types on the development set, although the finite sample leaves a wide confidence interval and two interface label errors remain. The ablation localizes the gate improvement to the negative evidence and safety layers.

The two EDA scenarios demonstrate the downstream meaning of the contract. Their frozen variables, constraints, outputs, selection rules, fresh solves, and physical reports support concrete nominal choices. They do not establish arbitrary COMSOL model generation, full manufacturing robustness, or physical agreement with fabricated devices. The six template executions establish traceability across representative mature builders and two scripting paths, not industrial coverage.

B. Limitations and submission gate

The main limitation is the absence of independently authored results. P2 tasks were written inside the project. P3 acquisition and method runs remain at zero, so the current manuscript is a format complete evidence bounded draft rather than a finished DAC result. Submission requires the frozen 24 task and 192 observation experiment, human verification of external identity and adjudication separation, and replacement of this paragraph with the generated independent table and failure analysis. If those records are not complete, the research manuscript should not be submitted as finished work.

The compiler uses a bounded lexicon and rule graph. New physics interfaces, geometry semantics, material models, contact, nonlinearities, bidirectional temperature dependence, and manufacturing constraints require explicit extensions and tests. COMSOL licensing and installation also limit reproduction. Another workstation must repeat a live template solve; copied MPH files cannot prove environment compatibility. No real temperature measurement currently supports claims about a fabricated physical system.

C. Threats to validity

Internal validity is limited by project authored P2 tasks and adjudication choices. The frozen inputs and raw responses reduce selective reporting but do not remove author bias. Construct validity depends on exact interface and coupling labels, which can penalize alternative valid decompositions; the final protocol therefore preserves explanations and failure categories for manual review. External validity is limited by a bounded COMSOL catalog, eight planned external families, one workstation, and two detailed EDA scenarios. Statistical conclusions remain task level and should not be extrapolated to industrial model complexity or unrestricted language.

Reproducibility is also conditional. Deterministic code, hashes, schemas, and compact results can be archived, but COMSOL and MATLAB licensing, version behavior, language model availability, and proprietary model weights may change. The protocol stores model identity, prompts, responses, and timing, and treats unavailable or invalid calls as failures. A later model version is a new experimental condition rather than a transparent replacement.

D. Engineering use and artifact review

The compiler is intended to sit before expensive model execution. A designer starts with a requirement and receives a review object containing the selected physics, couplings, study, requested observables, unresolved evidence, and the

reason for every block. An approved contract is then compiled through a mature builder rather than emitted as unrestricted solver code. The resulting bundle keeps the normalized request, contract, builder identity, executable source, solver log, MPH model, extracted values, preview, validation decisions, and file hashes under one task identifier. This structure lets a reviewer trace a reported temperature, displacement, or loss back to the dependency decision and the exact execution record that produced it.

Human review remains part of the authority boundary. The language model may explain a missing boundary, suggest a material property, or propose a repair, but it cannot override the static gate, mark a failed solver run as valid, or release a physical result. A changed contract creates a new execution record instead of silently replacing the previous evidence. This separation supports debugging and audit: physics selection errors, study errors, build errors, solver errors, and acceptance failures remain distinguishable. It also makes an incomplete task useful for method evaluation because a correct refusal and its diagnostic evidence can be scored without pretending that a valid model was built.

E. Frozen result insertion plan

The final external table will be generated directly from the sealed P3 report. It will include, for every method, task consensus exact physics and study accuracy, gate accuracy, false safe and false block counts, Wilson 95% intervals, completion time, interaction count, and repeat agreement where applicable. Pairwise McNemar tests will use the same task decisions, and paired bootstrap intervals will describe time differences. Per family results and the failure taxonomy will remain in the artifact even if space requires an aggregate table in the paper. Missing, invalid, or timed out model responses remain failures in the denominator.

No method, prompt, parser, template, threshold, or adjudicated label may change after the external task seal. The manuscript builder reads the machine report and checks its protocol, task, response, and seal hashes before permitting an independent result claim. If fewer than 24 accepted tasks, eight disjoint families, or 192 measured observations are available, the builder retains the explicit pending statement used in this draft. This policy prevents partial campaigns, favorable subsets, synthetic replacements, and post hoc repairs from entering the principal comparison. Human authors then inspect the generated table, every cited artifact, and the disclosure before approving a submission PDF.

F. Content Generated by AI

OpenAI Codex was used to help draft and edit the Abstract, Introduction, Related Work, Limitations, and Conclusion; to generate supporting Python and document formatting code; and to organize the reproducibility checklist. The system did not originate measured COMSOL outputs or external evaluation records. Human authors remain responsible for verifying every claim, citation, calculation, disclosure, and final sentence, and must revise this statement to match the actual submission process. No AI system is an author.

G. Conclusion

A traceable requirement to multiphysics workflow must decide both what to model and when not to proceed. The proposed compiler links observables to COMSOL dependencies, exposes missing and conflicting evidence, and preserves deterministic authority over construction, execution, repair, and validation. Controlled development results and two post freeze EDA scenarios support the mechanism and its downstream use. The frozen external protocol defines the remaining test. Independent performance claims will be added only after the sealed acquisition and all required runs are complete.

H. Reproducibility

The anonymous artifact package contains protocol JSON files, task and response schemas, evaluation code, tests, compact P1 and P2 reports, P3 readiness state, manuscript source, and SHA 256 hashes. It excludes credentials, personal paths, external identities, large MPH files, and unreviewed logs. The package records which claims are development, local execution, or pending external evidence.

REFERENCES

[1] Z. He et al., “ChatEDA: A Large Language Model Powered Autonomous Agent for EDA,” IEEE TCAD, 2024, arXiv:2308.10204.

[2] B. Liu et al., “LayoutCopilot: An LLM-Powered Multiagent Collaborative Framework for Interactive Analog Layout Design,” IEEE TCAD, vol. 44, no. 8, pp. 3126–3139, 2025, doi:10.1109/TCAD.2025.3529805.

[3] F. Wu et al., “Divergent Thoughts toward One Goal: LLM-based Multi-Agent Collaboration System for Electronic Design Automation,” Proc. NAACL, pp. 1615–1629, 2025.

[4] Y. Lu et al., “AutoEDA: Enabling EDA Flow Automation through Microservice-Based LLM Agents,” arXiv:2508.01012, 2025.

[5] “FlowFixer-EDA: A Tool-Feedback-Augmented Large Language Model Agent for Automated Diagnosis and Repair of Open-Source RTL-to-GDS Design Flow Failures,” IEEE Access, vol. 14, pp. 140412–140428, 2026, doi:10.1109/ACCESS.2026.3730896.

[6] T. Zhang, Z. Liu, Y. Xin, and Y. Jiao, “MooseAgent: A LLM Based Multi-agent Framework for Automating Moose Simulation,” arXiv:2504.08621, 2025.

[7] R. Deotale et al., “ALL-FEM: Agentic Large Language Models Fine-tuned for Finite Element Methods,” Comput. Methods Appl. Mech. Eng., 2026, doi:10.1016/j.cma.2026.118985.

[8] S. Adhikari, G. Noorsumar, and Ø. Jensen, “PDE-Agents: An LLM-Orchestrated Multi-Agent Framework for Automated Finite Element Simulations with Knowledge Graph-Augmented Reasoning,” arXiv:2606.07850, 2026.

[9] Z. Abulawi, M. Li, G. Giudicelli, Y. Liu, and C. Permann, “Deploying Frontier Agentic Technology in MOOSEnger, a Multiphysics-Capable AI Assistant,” arXiv:2608.15881, 2026.

[10] COMSOL AB, COMSOL Multiphysics Reference Manual, version 6.4, 2026.

[11] E. B. Wilson, “Probable Inference, the Law of Succession, and Statistical Inference,” J. Amer. Stat. Assoc., vol. 22, no. 158, pp. 209–212, 1927.

[12] Q. McNemar, “Note on the Sampling Error of the Difference between Correlated Proportions or Percentages,” Psychometrika, vol. 12, pp. 153–157, 1947.

[13] B. Efron and R. Tibshirani, An Introduction to the Bootstrap. Chapman and Hall/CRC, 1993.

[14] ACM and IEEE, “DAC 2027 Research Manuscript Submissions,” 2026. [Online]. Available: https://dac.com/2027/program/research-manuscript-submissions